



Departamento de Informática e Matemática Aplicada – DIMAp

Arquitetura Cloud Native

Prof. Nélio Cacho

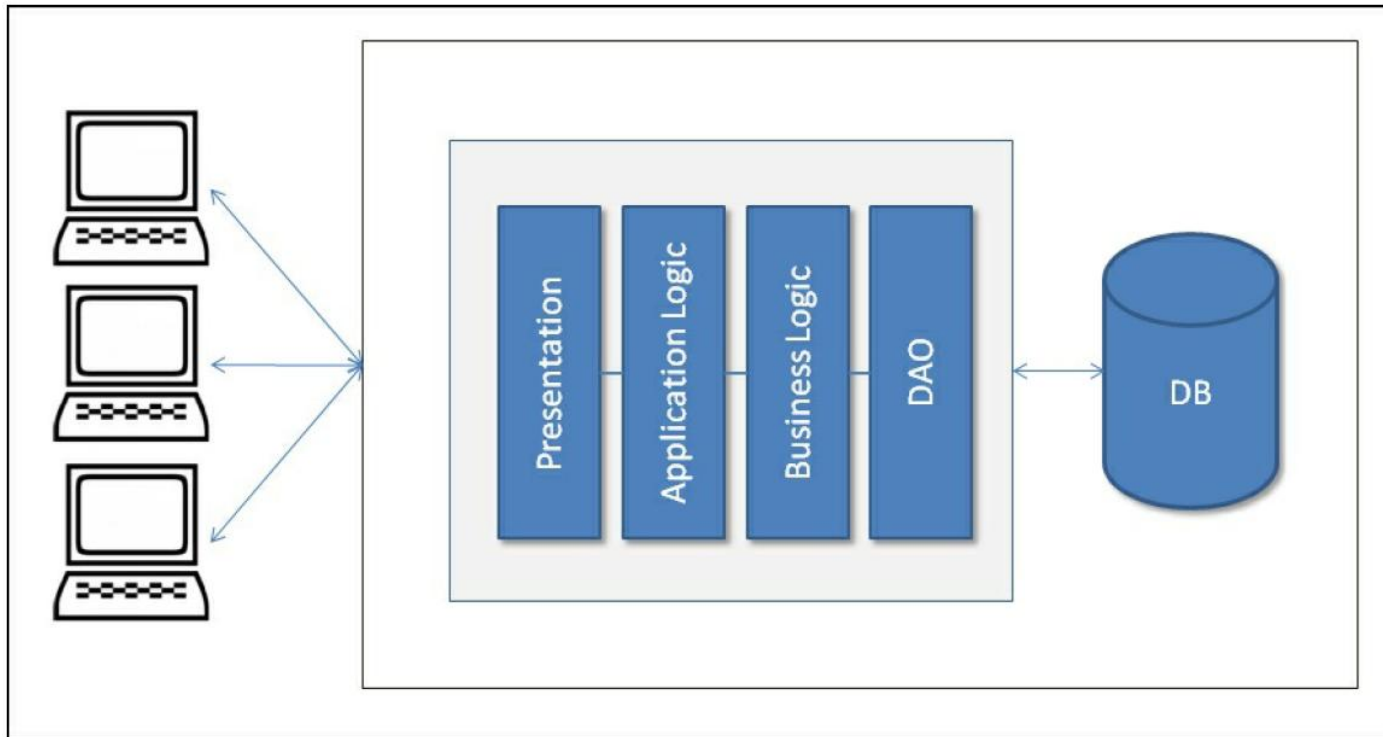
Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.
- Não está autorizada a gravação ou reprodução desta aula

Design Tradicional usando Monolítico



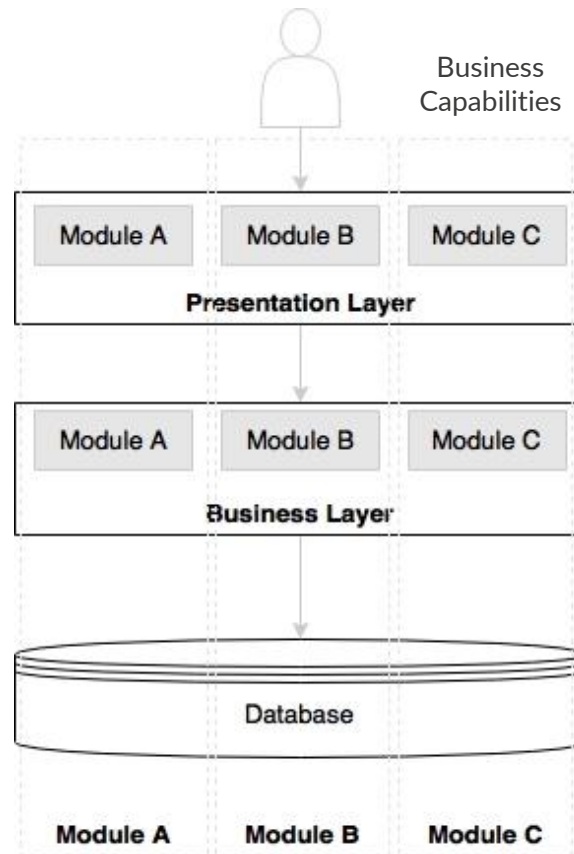
Problemas do Monolítico

- A manutenção na mesma base de código pode ser complicada devido a fusões que são difíceis de aplicar.
- Implementar novos recursos está se tornando cada vez mais complexo e pode levar mais tempo do que o esperado.
- Escalabilidade da aplicação.
- Implantar novos recursos sem impactar o que já está online é um desafio.
- Mudanças arquiteturais são sempre muito complexas.

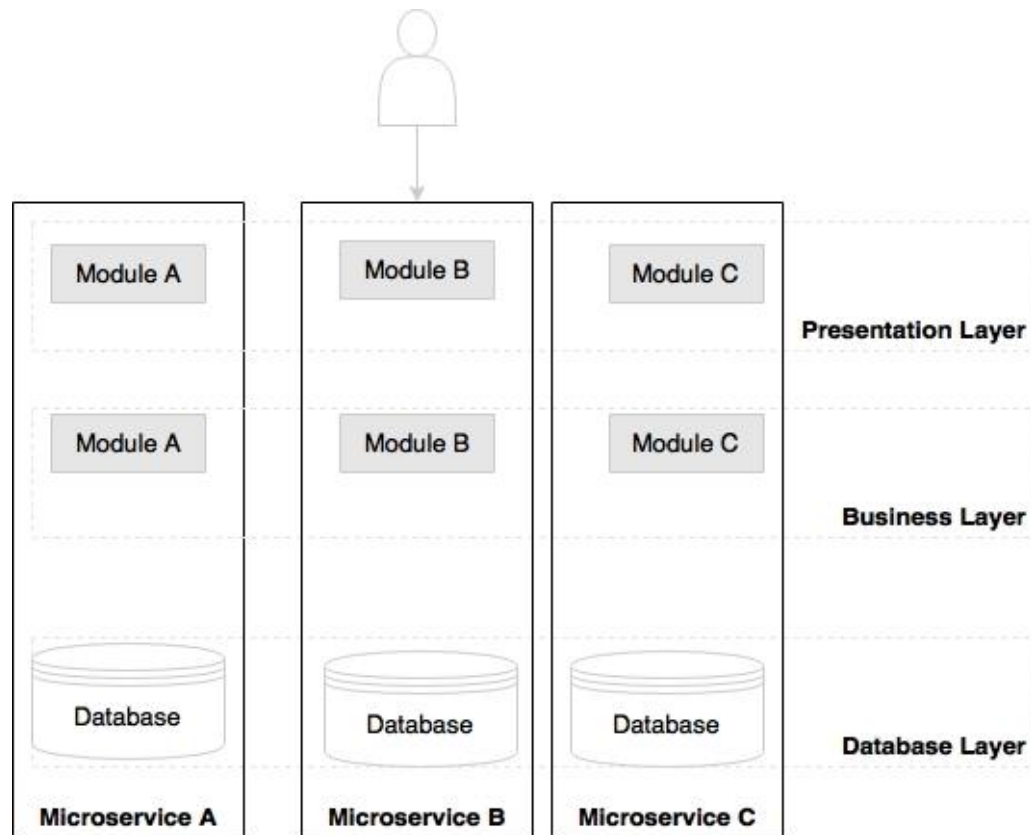
Microservice Concepts

"O estilo arquitetônico de [microserviços](#) é uma abordagem para desenvolver uma única aplicação como um conjunto de pequenos serviços, cada um executando em seu próprio processo e se comunicando por meio de mecanismos leves, frequentemente uma API de recursos HTTP."

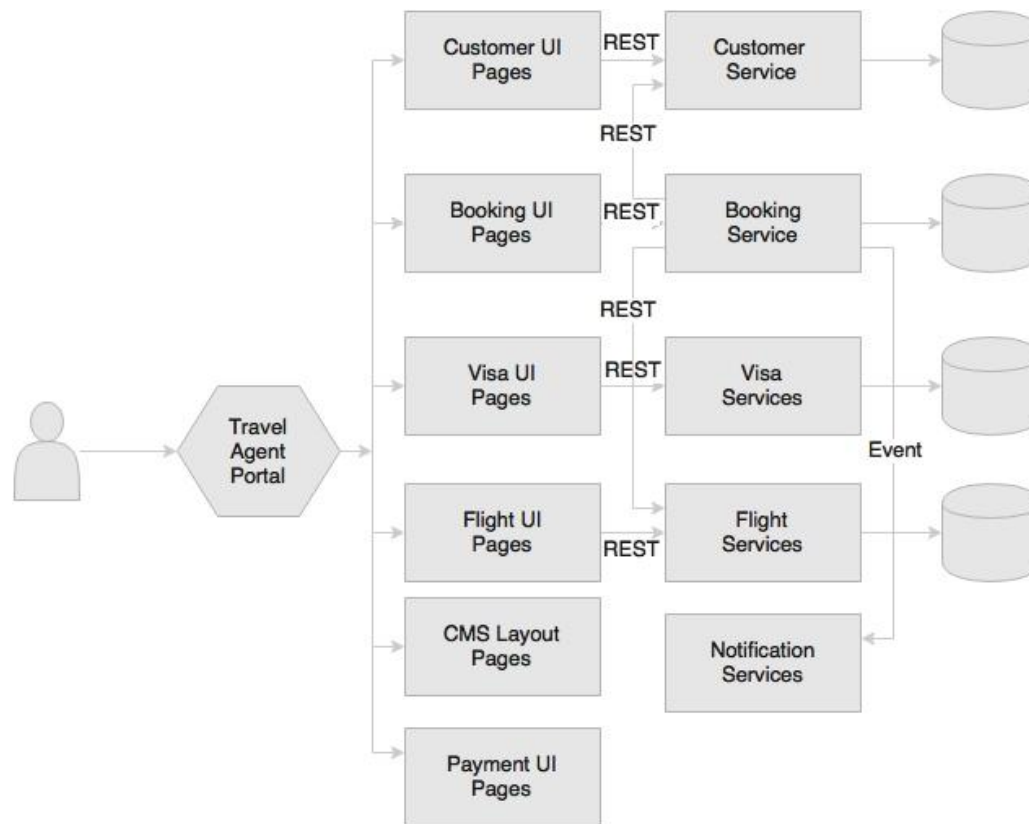
Migrando para Microserviços



Migrando para Microserviços



Exemplo de aplicação em Microserviços



Quais os problemas de usar Microserviços?

Maior complexidade para o desenvolvimento

- As coisas podem se tornar muito mais difíceis para os desenvolvedores.
- No caso em que um desenvolvedor deseja trabalhar em um recurso que pode abranger vários serviços, esse desenvolvedor precisa executá-los todos em sua máquina ou conectá-los. Isso muitas vezes é mais complexo do que simplesmente executar um único programa.
- Esse desafio pode ser parcialmente mitigado com ferramentas, mas à medida que o número de serviços que compõem um sistema aumenta, mais desafios os desenvolvedores enfrentarão ao executar o sistema como um todo.

Quais os problemas de usar Microserviços?

Maior complexidade para equipe de Infraestrutura

- Para equipes que não desenvolvem serviços, mas os mantêm, há uma explosão na complexidade potencial.
- Em vez de talvez gerenciar alguns serviços em execução, eles estão gerenciando dezenas, centenas ou milhares de serviços em execução.
- Há mais serviços, mais caminhos de comunicação e mais áreas de possível falha.

Docker containers

O Docker é um projeto de código aberto que automatiza a implantação de aplicativos dentro de contêineres de software, fornecendo uma camada adicional de abstração e automação da virtualização em nível de sistema operacional no Linux.[Source: en.wikipedia.org]

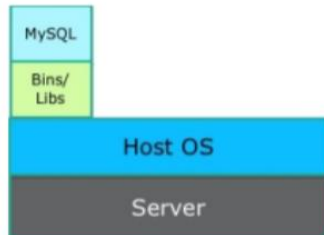
Fornece um camada padronizada para empacotar softwares: «*Build, Ship and Run Any App, Anywhere*»

[www.docker.com]

Como gerenciar muitos serviços?

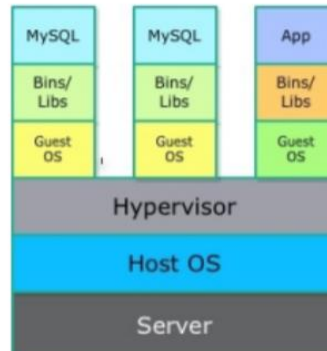
Bare Metal

Openstack ironic



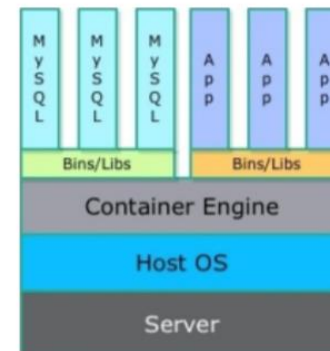
VM Based Host

Standard nova driver

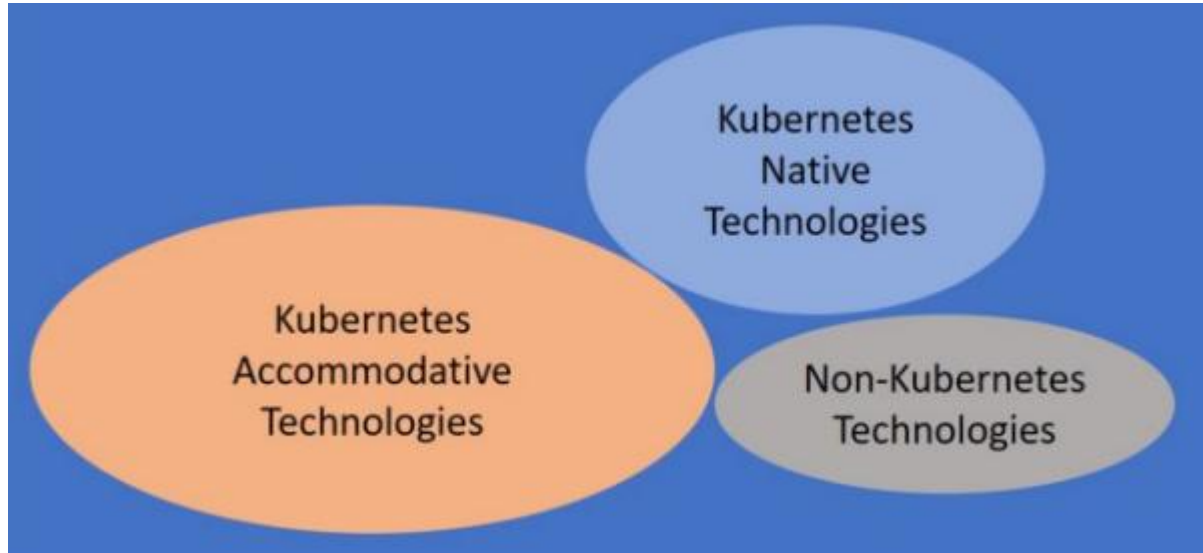


Container

Nova-docker driver



Como gerenciar muitos serviços? Orquestração!



Significado de “Cloud Native”

cloud native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds. Containers, service meshes, microservices, immutable infrastructure, and declarative APIs exemplify this approach.

These techniques enable loosely coupled systems that are resilient, manageable, and observable. Combined with robust automation, they allow engineers to make high-impact changes frequently and predictably with minimal toil.

—CNCF Cloud Native Definition v1.0

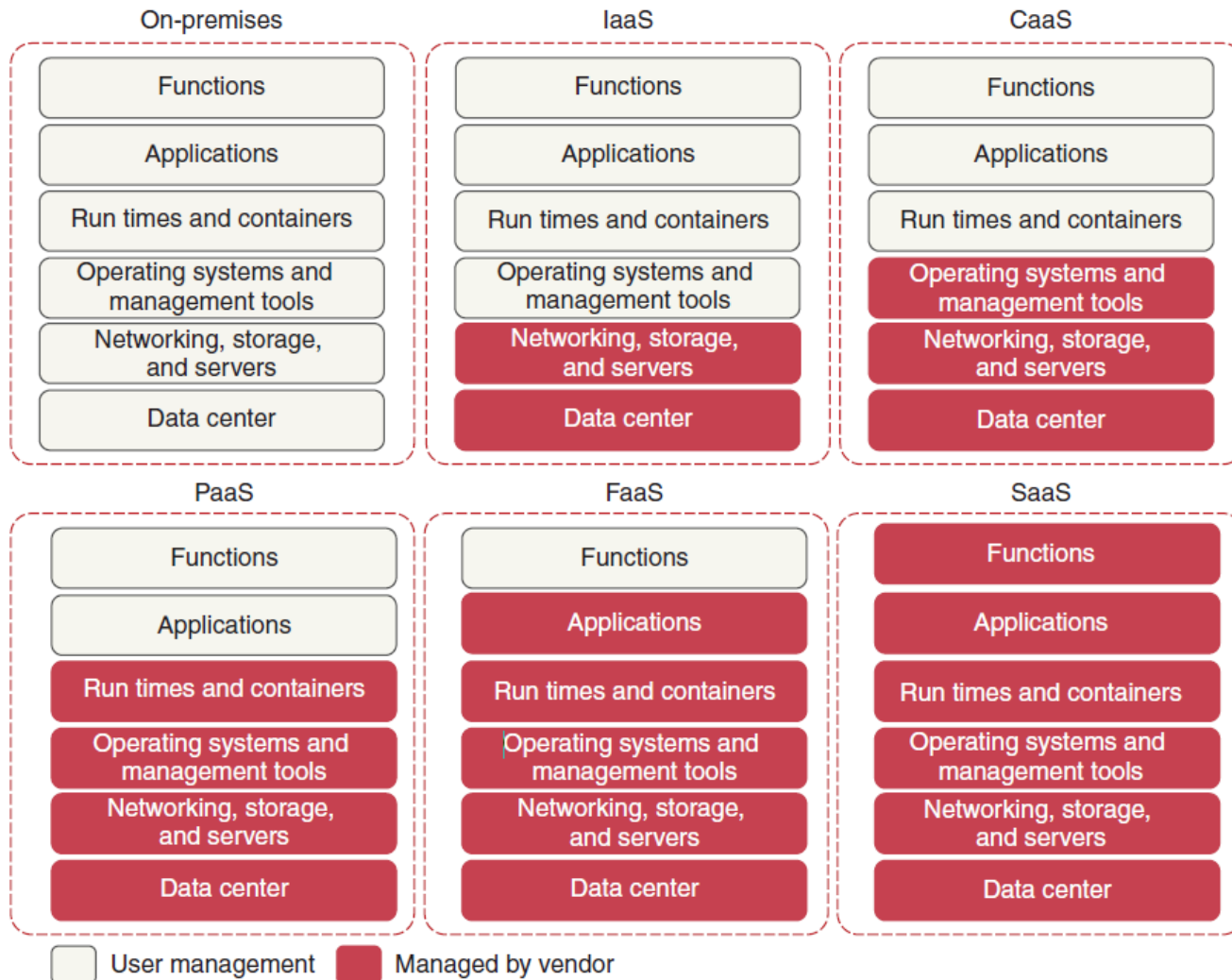
Os três Ps de Cloud Native

- **Plataformas.** Aplicações nativas da nuvem são executadas em plataformas baseadas em ambientes dinâmicos e distribuídos: as nuvens (pública, privada ou híbrida).
- **Propriedades.** Aplicações nativas da nuvem são projetadas para serem escaláveis, com baixo acoplamento, resilientes, gerenciáveis e observáveis.
- **Práticas.** As práticas em torno de aplicações nativas da nuvem incluem automação robusta combinada com mudanças frequentes e previsíveis: automação, entrega contínua e DevOps.

Tipos de Plataformas de Nuvem

Infrastructure Platform	Container Platform	Application Platform	Serverless Platform	Software Platform
IaaS	CaaS	PaaS	FaaS	SaaS
Platform: provides computing, storage, and networking resources.	Platform: provides container engine, orchestrator, and underlying infrastructure.	Platform: provides development and deployment tools, APIs, and underlying infrastructure.	Platform: provides the runtime, the whole infrastructure needed to run functions, and autoscaling.	Platform: provides both the software and the whole infrastructure needed to run it.
Consumer: provisions, configures, and manages servers, network, and storage.	Consumer: builds, deploys, and manages containerized workloads and clusters.	Consumer: builds, deploys, and manages applications.	Consumer: builds and deploys functions.	Consumer: consumes a service via a network.

Tipos de Plataformas de Nuvem



Properties

Application Properties

Scalability

Dynamically support increasing or decreasing workloads

Loose coupling

Components have minimal knowledge of each other

Resilience

Maintain level of service in face of adversities

Manageability

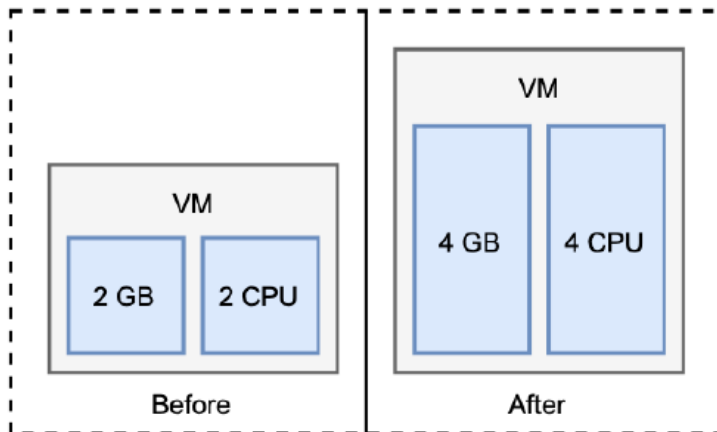
Level of control from the outside: update, configure, deploy

Observability

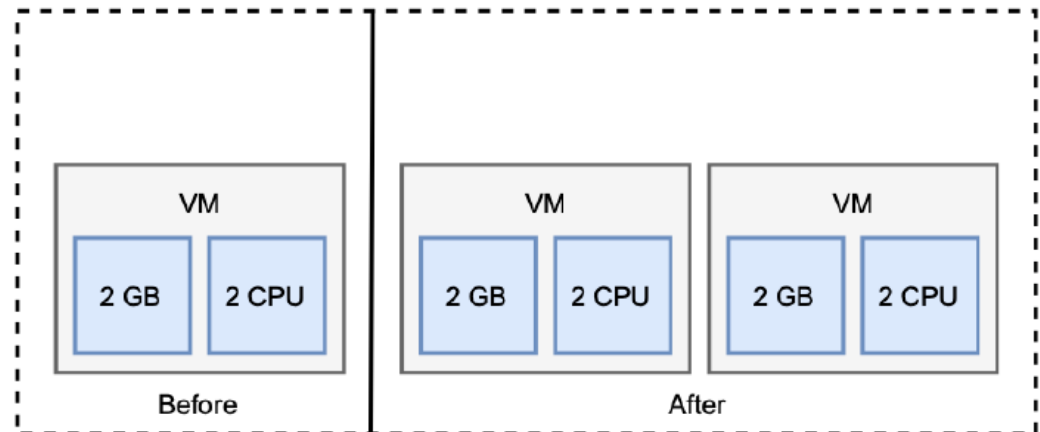
Knowledge about the internal state inferred from the outside

Scalability

Vertical Scalability



Horizontal Scalability



Práticas que suportam Cloud Native

Practices

Automation

Reproducible, efficient
and reliable systems

Continuous Delivery

Better and faster
software deliveries

DevOps

Culture of collaboration
between different roles

Service-based architecture

- Arquitetura baseada em serviço é composta por dois componentes:
 - **Serviço:** um componente que fornece qualquer tipo de serviço a outro componente. Serviços de aplicação são sem estado e são responsáveis por implementar qualquer tipo de lógica.
 - **Serviços de dados** são com estado e são responsáveis por armazenar qualquer tipo de estado. O estado é tudo o que deve ser preservado ao desligar um serviço e iniciar uma nova instância.
 - **Interação:** a comunicação dos serviços entre si para cumprir os requisitos do sistema.

Arquitetura baseada em serviços

Architectures

Applications Services

Loosely coupled, stateless,
independently deployable units

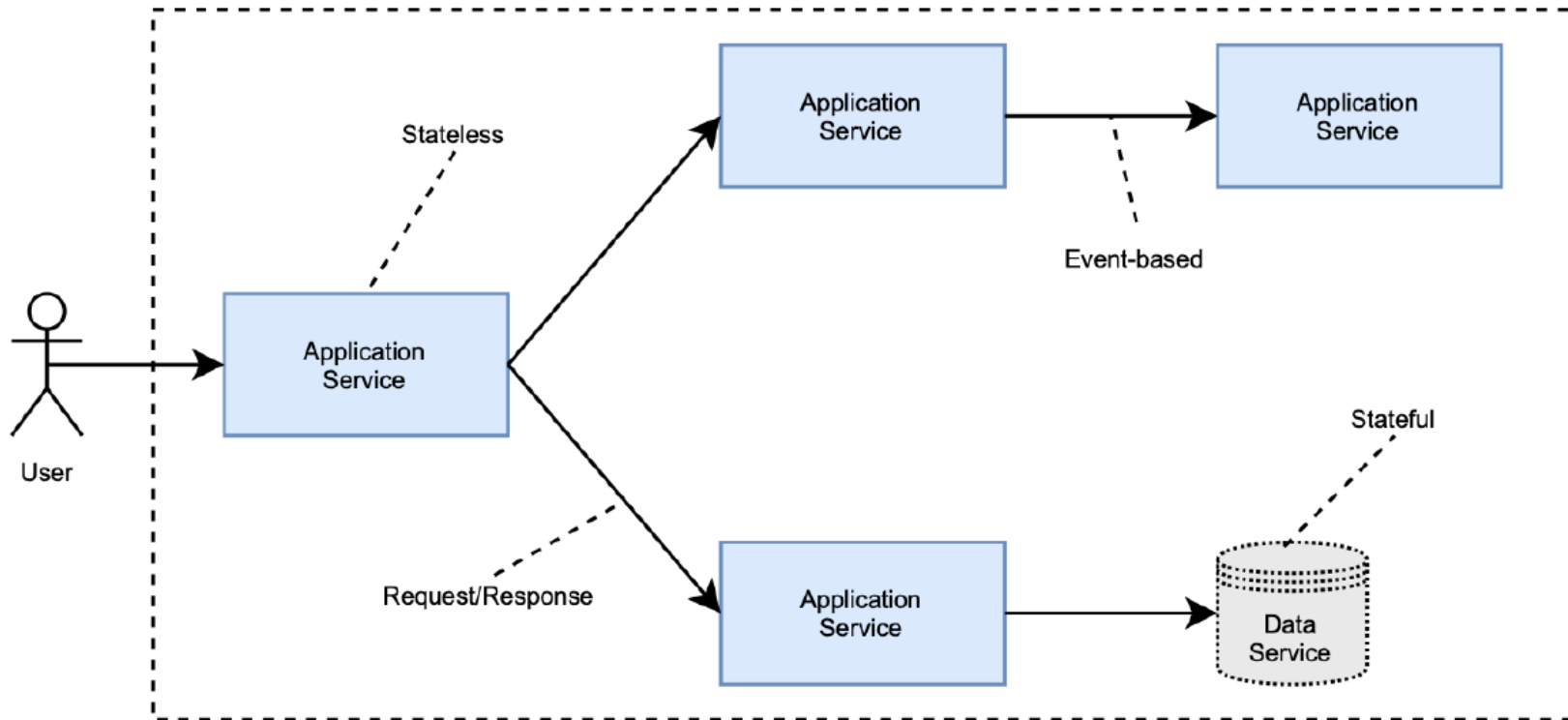
Data Services

Databases, messaging
systems, and other
stateful components

Interactions

Communication between
services, such as REST
RSocket, gRPC

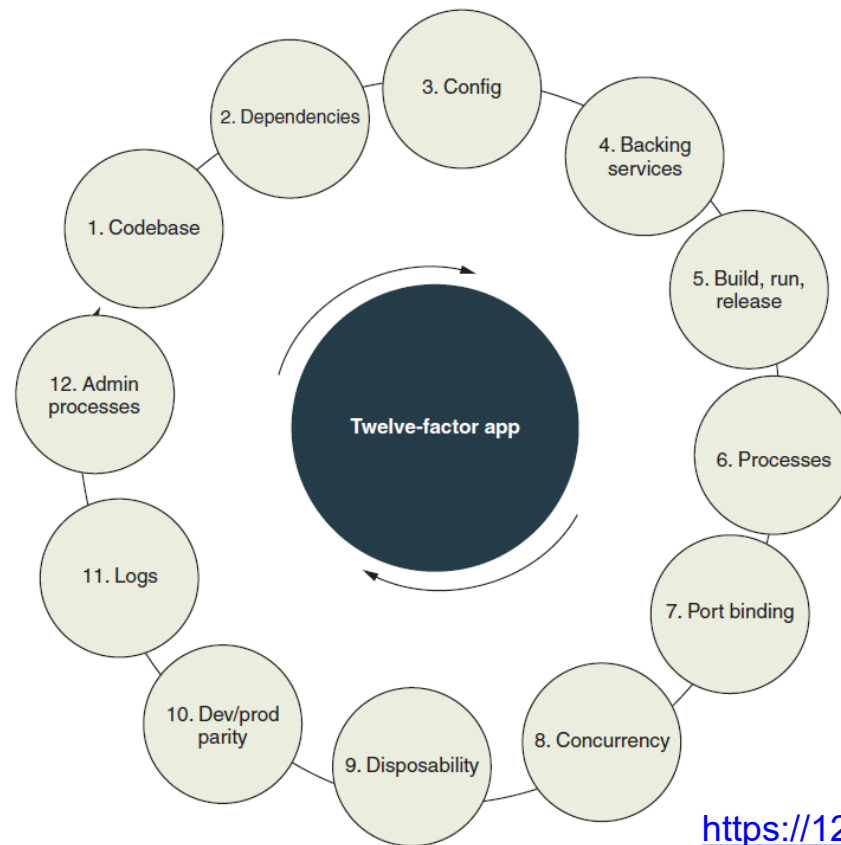
Arquitetura baseada em serviços



12 Fatores de boas práticas para aplicações Cloud Native

- A metodologia dos 12 fatores é um conjunto de doze melhores práticas para o desenvolvimento de aplicativos projetados para serem executados como serviços. Essa metodologia foi originalmente elaborada pela Heroku para aplicativos implantados como serviços em sua plataforma de nuvem, em 2011. Com o tempo, ela se mostrou genérica o suficiente para qualquer desenvolvimento de software como serviço (SaaS).
- Esses doze fatores são considerados diretrizes importantes para construir aplicativos modernos e escaláveis, garantindo portabilidade, facilidade de implantação e manutenção. Eles abordam várias áreas-chave, como configuração, dependências, escalabilidade, gerenciamento de processos e logs.
- Ao seguir a metodologia dos doze fatores, os desenvolvedores podem criar aplicativos que são flexíveis, fáceis de escalar e que se adaptam facilmente a diferentes ambientes de implantação. Isso promove a eficiência no desenvolvimento e a operação confiável de aplicativos como serviços em plataformas de nuvem.

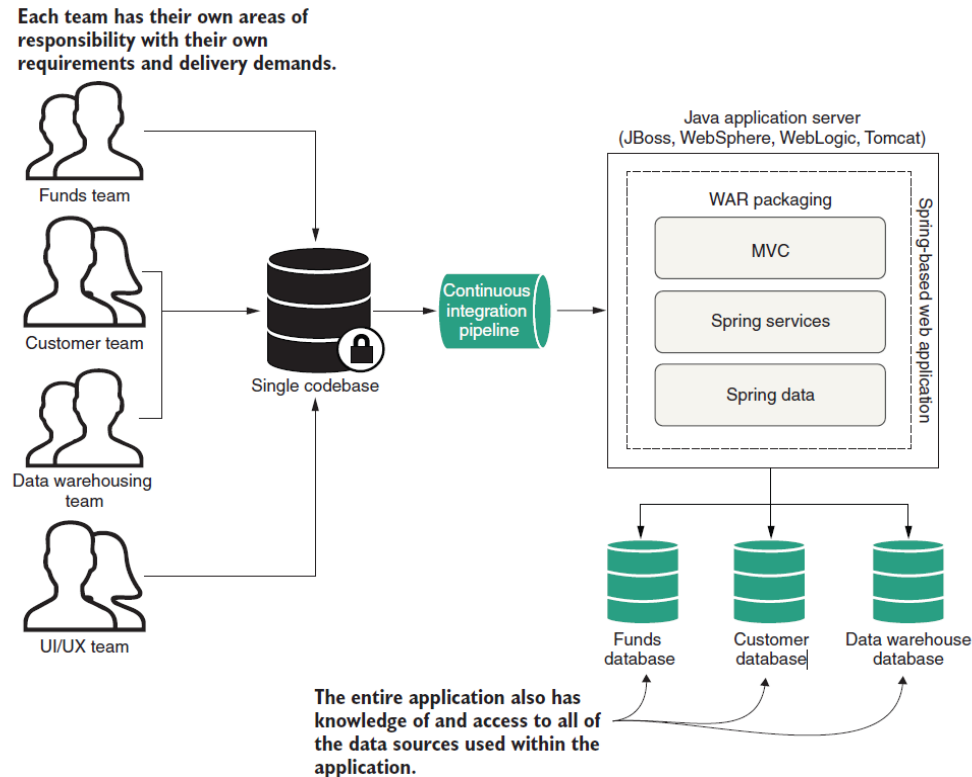
12 Fatores de boas práticas para aplicações Cloud Native



<https://12factor.net/>

1.Codebase

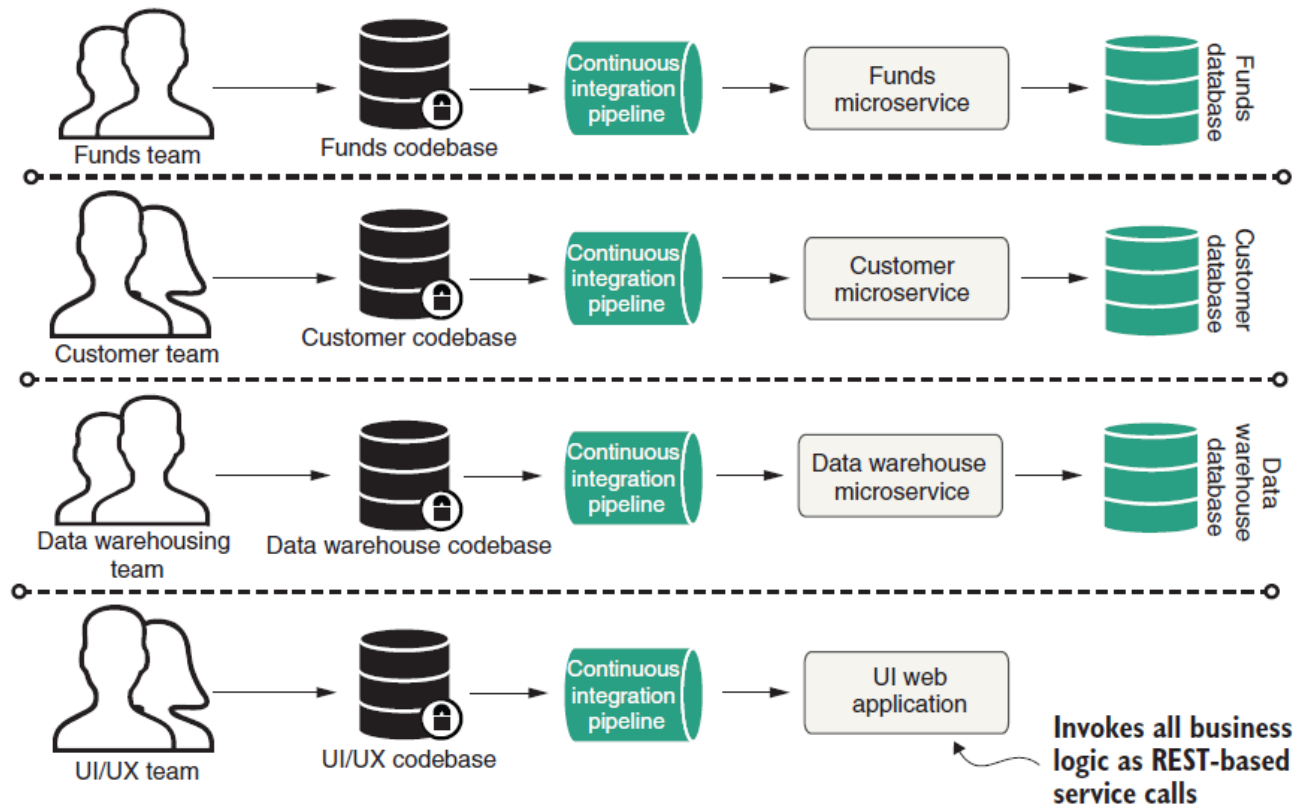
- O princípio afirma que um aplicativo deve ter seu próprio repositório de código e não deve compartilhar esse repositório com outros aplicativos.



1.Codebase

- Ao manter o código-fonte separado para cada microserviço, promove-se a modularidade, isolamento e independência.
- Cada aplicativo pode ter seu próprio controle de versão, ciclo de lançamento e processo de implantação, permitindo que equipes trabalhem em diferentes microserviços sem interferir no código-fonte uns dos outros.
- Ter um único repositório de código por aplicativo também ajuda na gestão de dependências, rastreamento de alterações e garante um processo de desenvolvimento claro e focado para cada microserviço individualmente.

1.Codebase



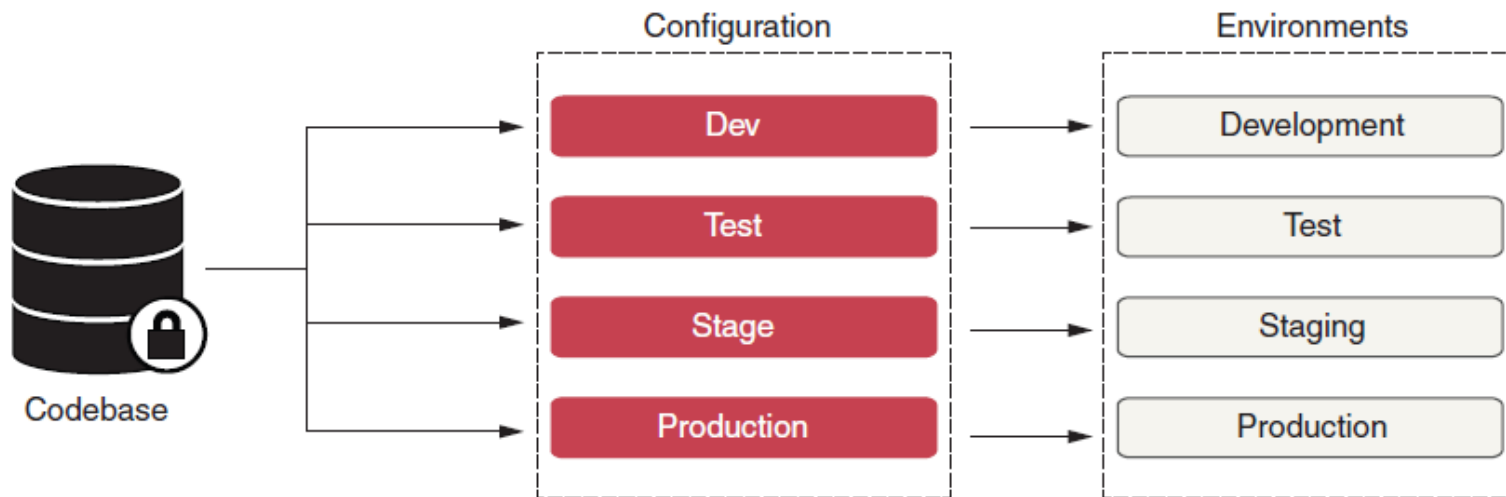
2.Dependências

- Essa melhor prática declara explicitamente as dependências que o seu microserviço deve utilizar por meio de ferramentas de construção, como o Maven ou o Gradle (Java).
- As dependências de terceiros em formato JAR devem ser declaradas usando **seus números de versão específicos**.
- Isso permite que você sempre construa seus microservices com a mesma versão da biblioteca.

3.Config

- Essa prática refere-se a como você armazena as configurações do seu aplicativo, especialmente as configurações específicas do ambiente.
- Nunca adicione configurações incorporadas ao seu código-fonte!
- Em vez disso, é melhor manter a configuração completamente separada do seu microserviço implantável.
- [Vamos ter uma aula sobre como fazer isso no Spring]

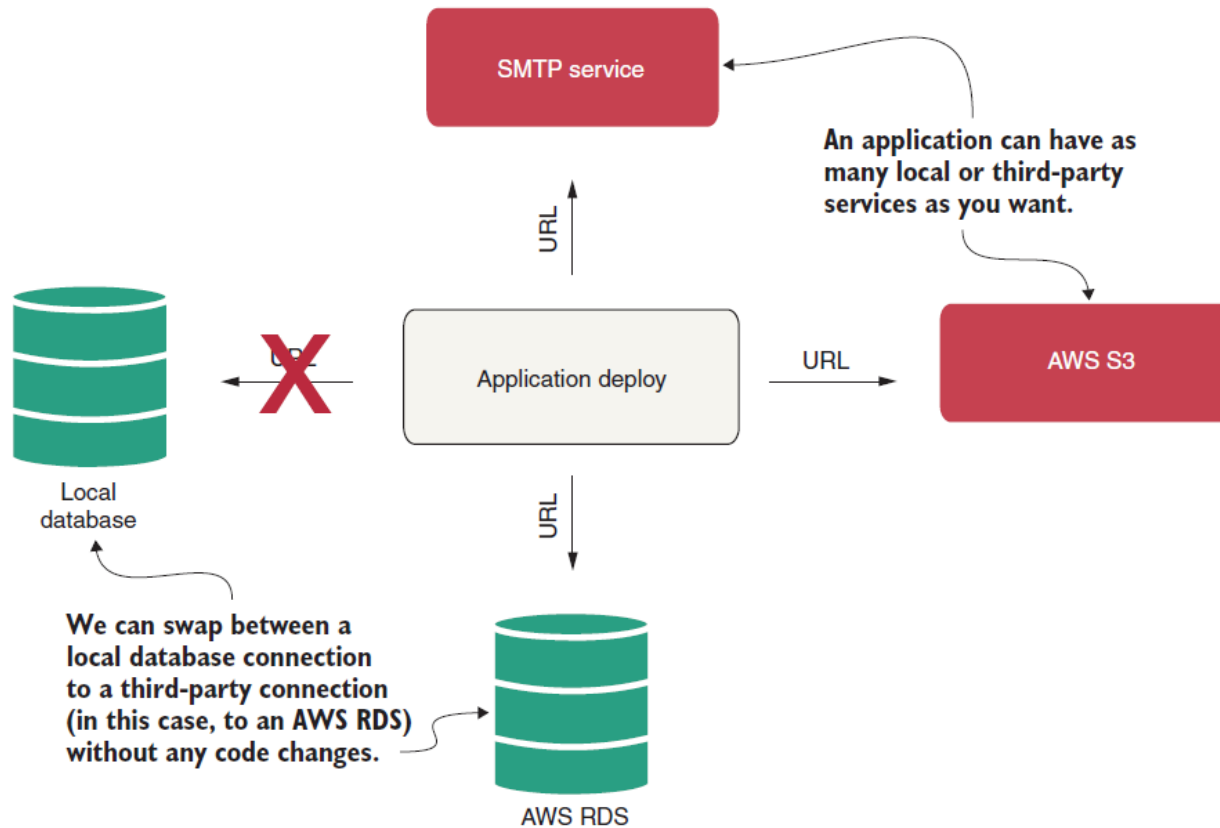
Config



Backing services

- Backing services são serviços dos quais o aplicativo depende para funcionar, como um banco de dados ou um serviço de mensagens.
- A boa pratica é que seu microserviço deve tratar todos esses serviços como recursos anexados. Isso significa efetivamente que não deve ser necessário fazer alterações no código para trocar um serviço de suporte compatível.
- A única alteração necessária deve ser feita nas configurações.
- **[Vamos ter uma aula sobre como fazer isso no Spring]**

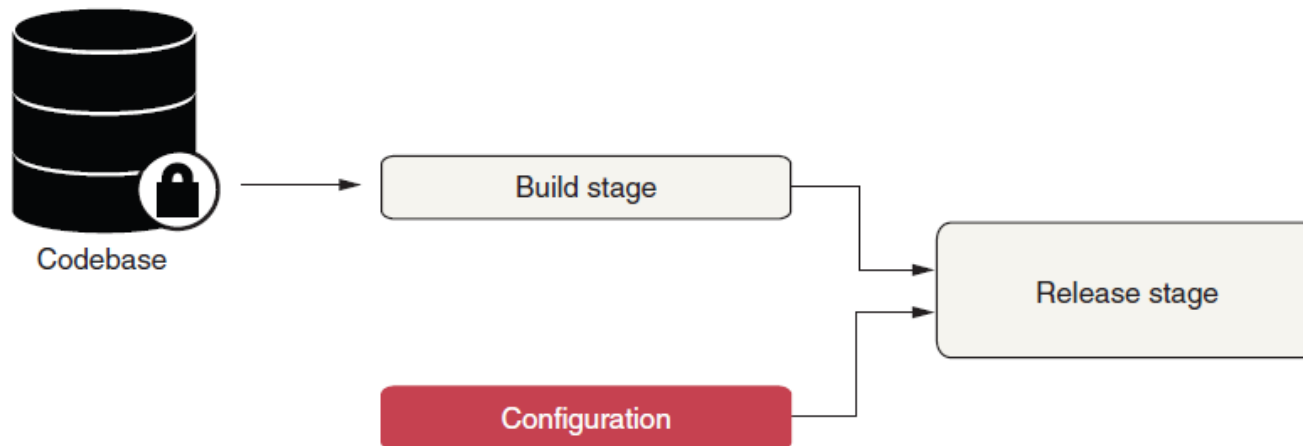
Backing services



Build, release, run

- Essa boa prática nos lembra de manter totalmente separadas as etapas de construção, lançamento e execução do deployment do microserviço.
- **Devemos ser capazes de construir microserviços que sejam independentes do ambiente em que são executados.**
- Uma vez que nosso código é construído, quaisquer alterações em tempo de execução precisam voltar para o processo de construção e serem redeployed.
- Um serviço construído é imutável e não pode ser alterado.

Build, release, run



Build, release, run

- **Build Stage:** Nesta etapa, pegamos a base de código, realizamos verificações estáticas e dinâmicas e, em seguida, geramos um pacote executável, como um arquivo JAR. Usando uma ferramenta como o Maven, isso é bastante trivial.

```
mvn clean compile test package
```

Build, release, run

- **Release Stage:** Esta é a etapa em que pegamos o pacote executável e o combinamos com as configurações corretas. Aqui, podemos usar o **Packer** com um provisionador como o **Ansible** para criar imagens do Docker.

```
packer build application.json
```

Build, release, run

- **Run Stage:** Por fim, esta é a etapa em que executamos o microserviço em um ambiente de execução. Se usarmos o Docker como o contêiner, podemos executar da seguinte forma:

```
docker run --name <container_id> -it  
<image_id>
```

Obviamente, não é necessário realizar manualmente essas etapas. É aí que o Jenkins se torna bastante útil com seu pipelines.

Processes

- Os 12 fatores demanda que um microserviço deve ser executado em um ambiente de execução como **processos sem estado**.
- Em outras palavras, eles não podem armazenar estado persistente localmente entre as solicitações.
- Eles podem gerar dados persistentes que devem ser armazenados em um ou mais serviços de suporte com estado.
- Em qualquer microserviço é comum termos vários endpoints expostos. Uma solicitação em qualquer um desses endpoints é completamente independente de qualquer solicitação feita antes dela.
- Por exemplo, se rastrearmos as solicitações do usuário na memória e usarmos essas informações para atender a solicitações futuras, isso viola as boas práticas dos 12 fatores.
- **Portanto, um microserviço que segue os 12 fatores não impõe restrições como sessões fixas (sticky sessions).** Isso torna esse tipo de aplicativo altamente portátil e escalável. Em um ambiente de execução em nuvem que oferece escalabilidade automática, esse comportamento é bastante desejável para os aplicativos.

Processes

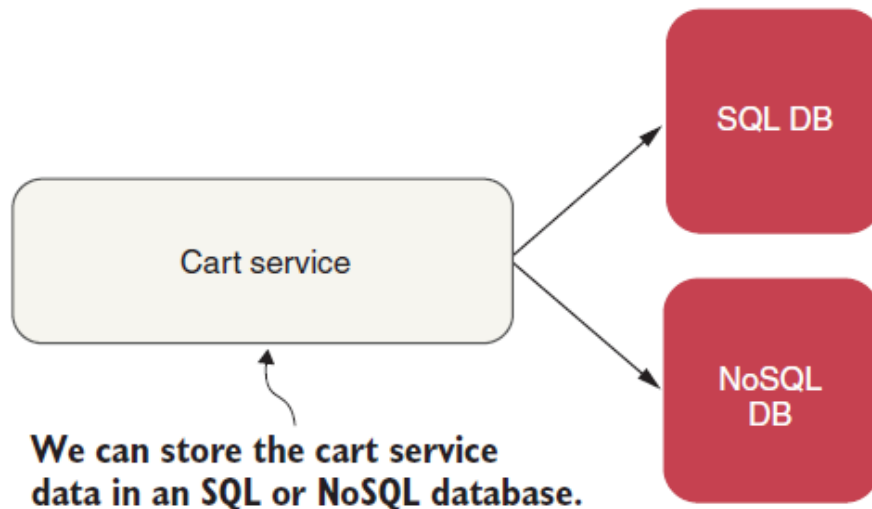


Figure 2.9 Stateless microservices don't store any session data (state) on the server. These services use SQL or NoSQL databases to store all the information.

Port binding

- Uma aplicação web tradicional em Java é desenvolvida como um WAR (Web Archive).
- Geralmente, é uma coleção de Servlets com dependências e espera um container de execução compatível, como o Tomcat.
- Por outro lado, os 12 fatores pedem que não haja dependência de tempo de execução desse tipo. O microserviço deve ser completamente autônomo e requerer apenas a chamada de um comando.

Port binding

- O Spring Boot já fornece um servidor de aplicativos embutido por padrão. Portanto, o arquivo JAR que mostramos anteriormente usando o Maven é totalmente capaz de ser executado em qualquer ambiente apenas tendo um tempo de execução Java compatível:
- `export SERVER_PORT=9090`
- `java -jar aplicacao.jar`

Aqui, nosso aplicativo simples expõe seus endpoints por meio de uma ligação HTTP em uma porta específica, como a 8080. Ao iniciar o aplicativo como fizemos acima, deverá ser possível acessar os serviços exportados por meio de HTTP.

Concurrency

- Esta boa pratica dos 12 fatores sugere que os microserviços dependam de processos para entregar escalabilidade.
- Uma aplicação deve ser escalada **através da multiplicação de processos**, não aumentando a complexidade dentro de um único processo.
- Isso significa efetivamente que os microserviços devem ser projetados para distribuir a carga de trabalho entre vários processos.
- No entanto, os processos individuais são livres para usar um modelo de concorrência, como Threads ou VirtualThreads, internamente.

Concurrency

- Imagine uma aplicação web com três tipos de tarefas:
 - **Servir requisições HTTP** (API ou site);
 - **Processar filas de mensagens** (como RabbitMQ, Kafka);
 - **Executar tarefas agendadas** (como relatórios ou limpeza de cache).
- Segundo o fator da **Concorrência**, você deve executar **um processo separado para cada tipo de trabalho**:

Tipo de tarefa	Tipo de processo	Exemplo de execução	
Servir HTTP	Web	<code>java -jar app.jar --type=web</code>	
Processar filas	Worker	<code>java -jar app.jar --type=worker</code>	
Tarefas agendadas	Cron	<code>java -jar app.jar --type=cron</code>	

Disposability

- Os processos podem ser encerrados de propósito ou devido a um evento inesperado.
- Em ambos os casos, um microserviço que segue os 12 fatores deve lidar com isso de maneira elegante.
- Em outras palavras, um processo deve ser completamente descartável, sem efeitos colaterais indesejados. Além disso, os processos devem iniciar rapidamente.
- Por exemplo, suponha que um endpoint é responsável por criar um novo registro de filme em um banco de dados.
- Agora, um aplicativo que lida com essa solicitação pode falhar inesperadamente. No entanto, isso não deve afetar o estado do aplicativo. Quando um cliente envia a mesma solicitação novamente, não deve resultar em registros duplicados.
- Em resumo, o microserviço deve expor serviços **idempotentes**.
- Esse é outro atributo muito desejável para um serviço destinado a implantações em nuvem. Isso oferece a **flexibilidade de interromper, mover ou iniciar novos serviços a qualquer momento, sem outras considerações**.

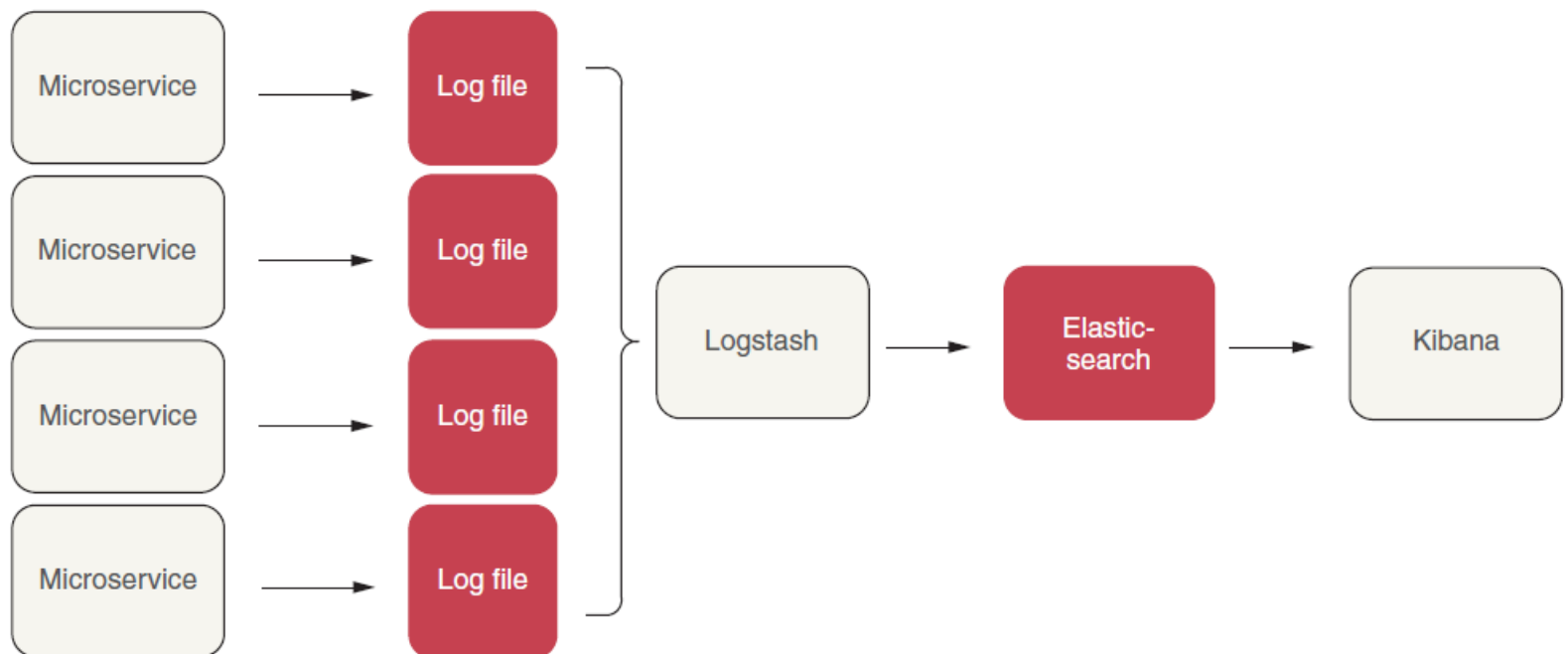
Dev/prod parity

- É comum que os aplicativos sejam desenvolvidos em máquinas locais, testados em outros ambientes e, finalmente, implantados em produção. Muitas vezes, esses ambientes são diferentes. Por exemplo, a equipe de desenvolvimento trabalha em máquinas com Windows, enquanto a implantação em produção ocorre em máquinas com Linux.
- A metodologia dos 12 fatores sugere manter a diferença entre o ambiente de desenvolvimento e o ambiente de produção o mínimo possível. Essas diferenças podem resultar de ciclos de desenvolvimento longos, diferentes equipes envolvidas ou uso de pilhas de tecnologia diferentes.
- Tecnologias como Spring Boot e Docker ajudam a diminuir significativamente essa diferença. Um aplicativo em um contêiner deve se comportar da mesma forma, não importa onde seja executado. Também devemos usar os mesmos serviços de suporte, como o banco de dados.

Logs

- Os logs são dados essenciais que um microserviço gera durante sua vida útil.
- Eles fornecem informações valiosas sobre o seu funcionamento.
- Seguindo os 12 fatores, o microserviço deve se separar da geração e processamento de logs. Ele simplesmente grava esses eventos na saída padrão do ambiente de execução. A captura, armazenamento, curadoria e arquivamento dessa sequência devem ser tratados pelo ambiente de execução.
- Existem várias ferramentas disponíveis para esse propósito. Podemos usar o SLF4J para lidar com o registro de logs de forma abstrata em nosso aplicativo. Além disso, podemos usar uma ferramenta como o Fluentd para coletar a sequência de logs dos aplicativos e serviços de suporte.
- Podemos alimentar esses logs no Elasticsearch para armazenamento e indexação. Por fim, podemos gerar painéis significativos para visualização no Kibana.

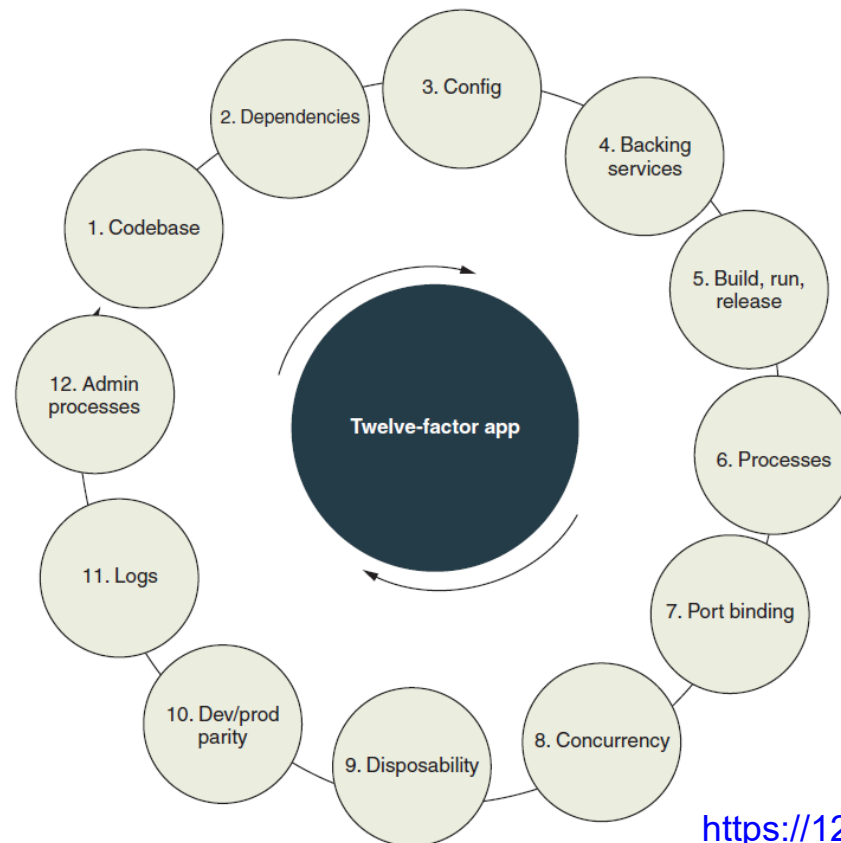
Logs



Admin processes

- Muitas vezes, precisamos realizar tarefas pontuais ou procedimentos de rotina durante a execução do microserviço. Por exemplo, corrigir registros incorretos. Existem várias maneiras de fazer isso. Como pode não ser necessário com frequência, podemos escrever um pequeno script para executá-lo separadamente de outro ambiente.
- A metodologia dos 12 fatores sugere fortemente manter esses scripts administrativos junto com o código-fonte do microserviço.
- Ao fazer isso, eles devem seguir os mesmos princípios que aplicamos ao código-fonte principal do aplicativo. Também é aconselhável usar uma ferramenta REPL integrada ao ambiente de execução para executar esses scripts em servidores de produção.
- Por exemplo, podemos escrever uma pequena função(Spring Command line) em Java para ler uma lista de filmes de um arquivo e salvá-los em lote no banco de dados.

Twelve-factor application best practices



<https://12factor.net/>

Spring Cloud

- O Spring Cloud fornece ferramentas para desenvolvedores construírem rapidamente alguns dos padrões comuns em arquiteturas Cloud Native.
- O Spring Cloud concentra-se em oferecer uma boa experiência "pronta para uso" para casos de uso típicos e um mecanismo de extensibilidade para abranger outros.
 - Distributed/versioned configuration
 - Service registration and discovery
 - Routing
 - Service-to-service calls
 - Load balancing
 - Circuit Breakers
 - Distributed messaging

Evolução do Spring Cloud

- Desde o lançamento do Spring Cloud Greenwich (V2.1) em janeiro de 2019, algumas das ferramentas da Netflix foram colocadas em modo de manutenção no Spring Cloud.

Current component	Replaced by
Netflix Hystrix	Resilience4j
Netflix Hystrix Dashboard/Netflix Turbine	Micrometer and monitoring system
Netflix Ribbon	Spring Cloud load balancer
Netflix Zuul	Spring Cloud Gateway

Spring Cloud numa Arquitetura Cloud Native

